Workshop 2: Advanced data wrangling: Extracting ecological signals from noisy systems

2.1 Workshop overview

Welcome to Workshop 2 of R for Marine Science! Building on our skills learned in Programming Fundamentals, our first session continued to develop the foundational mechanics of data wrangling. In this workshop, we are going to continue building our toolbox to help us extract biological signals from inherently noisy ecological and organismal systems. Today we will cover the principles of "tidy data" and master the tidyverse tools required to reshape, lengthen, and widen your data frames. We will also cover relational architecture—teaching you how to use mutating joins (`dplyr`) to merge messy biological datasets with spatial and temporal metadata. Along the way, we will tackle the inevitable string mismatches and date-time formatting errors using `stringr` and `lubridate`.

AI policy: As we navigate this added complexity, this workshop transitions into an optional “AI-On” environment. You are encouraged to utilise GitHub Copilot and conversational AI to help build your data wrangling pipelines. However, with this power comes the responsibility of ***Monitoring your AI Agents*** (emphasis all ours!!). To do this you need to critically evaluate, line-by-line, what your AI agents are generating to ensure your ecological outputs remain scientifically, programmatically and mathematically robust. You are still responsible for your work, and need to understand and explain (comments) what it is doing.

WIth that said, let’s get into it!

2.2 Tidying data

Tidy data are happy data! Or rather, tidy data are useful data. So we are going to learn how to wrangle our data into **tidy formats** that can be used in the tidyverse and beyond. Getting our data into a tidy format streamlines how we analyse it.

In this section, you will be introduced to tidy data and the accompanying tools in the **tidyverse**, which you should now be quite familiar with. Before starting this section, make sure the tidyverse is loaded.

```
library(tidyverse)
```

There are many ways to display a given data set, but not every way is easy to use for analysis. For example, the format of a field datasheet might be convenient for data collection and entry, but it might not be a useful way to display the data when you go to analyse it. The process of rearranging your data to a format more appropriate for analysis is the process of “tidying.”

Let’s look at an example below:

```
table1
#> # A tibble: 6 × 4
#>   country      year  cases population
#>   <chr>       <int>  <int>      <int>
#> 1 Afghanistan  1999    745   19987071
#> 2 Afghanistan  2000   2666   20595360
#> 3 Brazil       1999  37737  172006362
#> 4 Brazil       2000  80488  174504898
#> 5 China        1999 212258 1272915272
#> 6 China        2000 213766 1280428583
table2
#> # A tibble: 12 × 4
#>   country      year type           count
#>   <chr>       <int> <chr>          <int>
#> 1 Afghanistan  1999 cases            745
#> 2 Afghanistan  1999 population  19987071
#> 3 Afghanistan  2000 cases           2666
#> 4 Afghanistan  2000 population  20595360
#> 5 Brazil       1999 cases          37737
#> 6 Brazil       1999 population 172006362
#> # ... with 6 more rows
table3
#> # A tibble: 6 × 3
#>   country      year rate
#> * <chr>       <int> <chr>
#> 1 Afghanistan  1999 745/19987071
#> 2 Afghanistan  2000 2666/20595360
#> 3 Brazil       1999 37737/172006362
#> 4 Brazil       2000 80488/174504898
#> 5 China        1999 212258/1272915272
#> 6 China        2000 213766/1280428583
```

Each table displays the exact same dataset, but only `table1` is “tidy.” Why? Do you see the differences between these tables? Let’s go over what makes a tidy dataset and why you ***always*** should strive to get your data into a tidy format.

How we make our dataset tidy is by following three interrelated rules.

1. Each variable must have its own column.
2. Each observation must have its own row.
3. Each value must have its own cell.

Here is a figure from the R4DS textbook to help you visualise these rules:

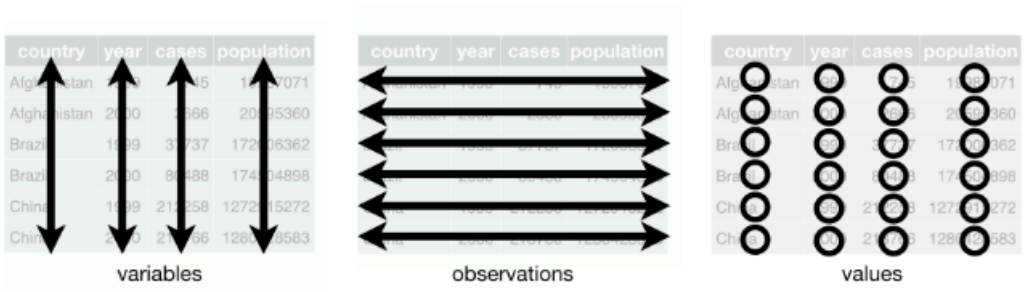


Figure 12.1: Following three rules makes a dataset tidy: variables are in columns, observations are in rows, and values are in cells.

Based on these rules, do you see why `table1` is tidy and the others aren't?

These three rules are interrelated because it's impossible to only satisfy two of the three. That interrelationship leads to an even simpler set of practical instructions:

1. Put each dataset in a tibble (special dataframe)
2. Put each variable in a column.

Now, why do we care about having tidy data? We said before, tidy data is useful data and here's why:

1. Having a consistent data structure makes it easier to learn the tools that work with it, and
2. Having your variables in columns allows R to use its ability to work with vectors of values. This makes *transforming* tidy data a smoother process.

All packages in the tidyverse are designed to work with tidy data. Including ggplot2.

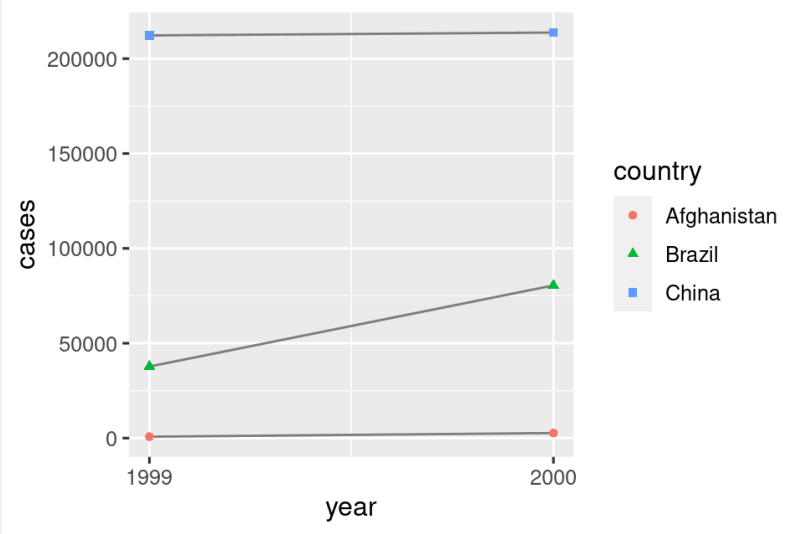
Here are some examples of how you might work with tidy `table1` from the previous example using some skills we're about to learn. The key here is to note that filtering data, summarising data and using functions like group or color in ggplot2, is possible with a dataframe in this format, but *impossible* if it's not in this format.

```
# Compute rate per 10,000
table1 %>%
  mutate(rate = cases / population * 10000)
#> # A tibble: 6 × 5
#>   country      year  cases population  rate
#>   <chr>       <int>  <int>      <int> <dbl>
#> 1 Afghanistan  1999    745   19987071 0.373
#> 2 Afghanistan  2000   2666   20595360 1.29
#> 3 Brazil       1999  37737  172006362 2.19
#> 4 Brazil       2000  80488  174504898 5.61
```

```
#> 5 China        1999 212258 1272915272 1.67
#> 6 China        2000 213766 1280428583 1.67

# Compute cases per year
table1 %>%
  count(year, wt = cases)
#> # A tibble: 2 × 2
#>    year      n
#>   <int>  <int>
#> 1  1999 250740
#> 2  2000 296920

# Visualise changes over time
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), colour = "grey50") +
  geom_point(aes(colour = country))
```



Understanding whether your data frame structure is optimal (or tidy) is a **fundamental skill for a data scientist.** We rarely underline and bold in these workbooks, but cannot stress this enough - once you master your understanding of how to best structure a data frame, everything else in R will become easy (well, *easier*).

2.3 Pivoting data to make it tidy

Even though tidy data is super handy, most of the data you'll encounter will likely be untidy. Many people aren't familiar with the concept of tidy data, and the format in which data is

collected is not always done with future analyses in mind. This means that with most data, some amount of tidying will be needed before you can commence analysis.

So let's dive in. What should we do with the dataset you've collected? How should we *transform* it to get it into a structure where we can start to do things to it?

The **first step** in tidying the data is to *understand what each variable and observation actually means.* Sometimes this is obvious and sometimes you'll need to consult with the person(s) who collected the data.

And often the person who knows the most about the data is *YOU!* So while learning how to tidy data in R is critical, the way in which you enter your data into excel is also vital!

The understanding of data structures here can translate into better data entry, and I think, can be another one of those pieces of knowledge that will *change your life* in the future!

Once you understand the data you're looking at, the **second step** is to resolve one of the two common problems with untidy data. These are:

1. One variable is spread across multiple columns
2. One observation is scattered across multiple rows

Hopefully your dataset will only have one of these problems but sometimes you may encounter both.

To fix these we will **pivot** our data (i.e. move it around) into tidy form using two functions in `tidyr`: `pivot_longer()` to lengthen data and `pivot_wider()` to widen data. Let's explore these functions a bit further.

2.4 Lengthening datasets

We'll start with pivoting our data frame *longer* because this is the most common tidying issue you will likely face within a given dataset. `pivot_longer()` makes datasets "longer" by increasing the number of rows and decreasing the number of columns, solving those common problems of data values in the variable name (e.g wk1, wk2, wk3, etc.).

Let's visualize this in the image below. The table on the right (`table4`) needs to be wrangled into 'long' format (represented by the table on the left) in order to satisfy the 3 rules of tidy data (section **4.4**). The issue here is that we want to group our data by year, or use it as a factor, and ggplot2 and most statistical packages cannot do this with *data* in columns. Only variables should be in columns. Therefore, including the variable we want to group by (year in the below) we pass the variable name to any grouping function, like `facet_wrap()`.

Using `pivot_longer()` splits the dataset by column, and reformats it into the tidy format of observations as rows, columns as variables and values as cell entries. The dataset is now longer (more rows, at left) than the 'wide format' data on the right.

Figure 12.2: Pivoting `table4` into a longer, tidy form.

Let's see this in action by using `pivot_longer()` in an example given by R4DS. The billboard dataset records the billboard rank of songs in the year 2000:

```
billboard
#> # A tibble: 317 × 79
#>   artist       track               date.entered   wk1   wk2   wk3   wk4   wk5
#>   <chr>        <chr>               <date>       <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 2 Pac        Baby Don't Cry (Ke... 2000-02-26      87    82    72    77    87
#> 2 2Ge+her      The Hardest Part O... 2000-09-02      91    87    92    NA    NA
#> 3 3 Doors Down Kryptonite          2000-04-08      81    70    68    67    66
#> 4 3 Doors Down Loser               2000-10-21      76    76    72    69    67
#> 5 504 Boyz     Wobble Wobble       2000-04-15      57    34    25    17    17
#> 6 98^0         Give Me Just One N... 2000-08-19      51    39    34    26    26
#> # ℹ 311 more rows
#> # ℹ 71 more variables: wk6 <dbl>, wk7 <dbl>, wk8 <dbl>, wk9 <dbl>, ...
```

Remember our **first step** in evaluating a dataset? It is to understand what each *observation* means, so we can be sure they are represented appropriately as rows.

In this dataset, each observation is a song. The first three columns (`artist, track` and `date.entered`) are variables that describe the song. Then we have 76 columns (`wk1-wk76`) that describe the rank of the song in each week. Here, the column names are one variable (the `week`) and the cell values are another (the `rank`). To tidy the billboard dataset we will use `pivot_longer().`

This is the case where actual data *values* (wk1, wk2 etc.) are in the column name, with each observation (row of data) being a *song*. We need to have the data in a format where each row is an *observation* (so-called long format).

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank"
  )
#> # A tibble: 24,092 × 5
#>    artist track                   date.entered week   rank
#>    <chr>  <chr>                   <date>       <chr> <dbl>
#>  1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
#>  2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
#>  3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
#>  4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
#>  5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
#>  6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
#>  #> # ℹ 24,082 more rows
```

As you can see in the above code snippet, there are three key arguments to the pivot_longer() function:

1. `cols` which specifies the columns you want to pivot (the ones that aren't variables). *Note: you could either use* `!c(artist, track, date.entered)` *OR* `starts_with('wk')` *because the* `cols` *argument uses the same syntax as* `select()`.
2. `names_to` which names the variable stored in the column names. We chose to name that variable `week`.
3. `values_to` which names the variable stored in the cell values that we named `rank`

Note that in the code `"week" and "rank"` are quoted because they are new variables that *we are creating*, they don't exist yet in the data when we run the `pivot_longer()` call.

Notice the NA values in the output above? It looks like "Baby Don't Cry" by 2 Pac was only in the top 100 for 7 out of 76 weeks. Therefore, when we lengthened the data, the weeks where it wasn't on the charts became 'NA.' These NA's were forced to exist because of the structure of the dataset not because they are actually unknown. Therefore, we can simply ask `pivot_longer` to remove them by adding the argument `values_drop_na = TRUE` as shown below:

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank",
    values_drop_na = TRUE
```

```
  )
#> # A tibble: 5,307 × 5
#>   artist track                   date.entered week   rank
#>   <chr>  <chr>                   <date>       <chr> <dbl>
#> 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
#> 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
#> 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
#> 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
#> 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
#> 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
#> # ℹ 5,301 more rows
```

There are now far fewer rows in total, indicating that a heap of NAs were dropped.

Congratulations! Our data is now tidy!

However, do note that there are still some things we could do to improve the format to make future computation easier. Such as converting some of our values from strings to numbers using `mutate()` *and* `parse_number()`. *See the end of section 6.3.1 in the textbook.*

Pivoting longer - example

We've just seen how pivoting can help reshape our data but let's look further into what pivoting actually does to our data. We will start with a simple dataset and once again follow an example from R4DS. Note the use of the term "tribble" here (not the same as tibble) but also a type of dataframe that allows us to construct small tibbles by hand. Don't get too caught up in this, simply follow along with the example.

```
df <- tribble(
  ~id,  ~bp1, ~bp2,
   "A",  100,  120,
   "B",  140,  115,
   "C",  120,  125
)
```

Here, all we have done is created a dataset called '**df'** with 3 variables and their associated values.

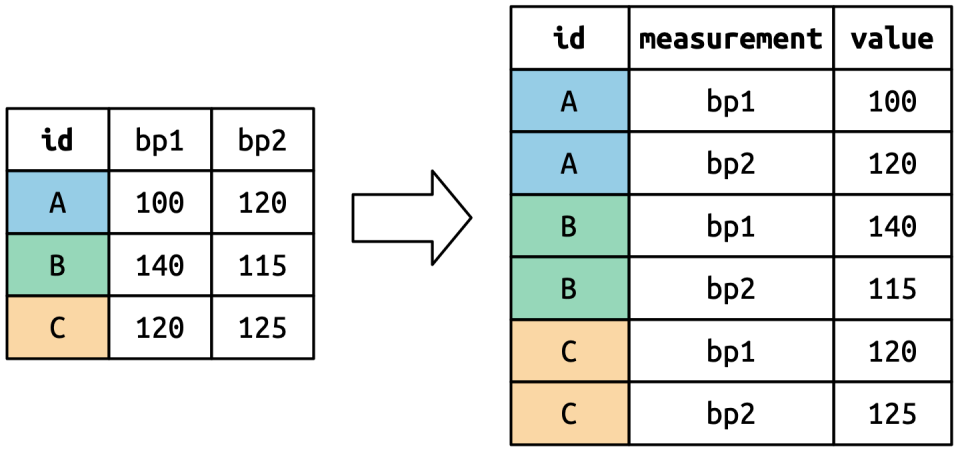
However, we want our new (tidy) dataset to have three variables:

1. `id` (which already exists)
2. `measurement` (the column names)
3. `value` (the cell values)

To make this happen we need to *pivot* **df** *longer:*

```
df |>
  pivot_longer(
    cols = bp1:bp2,
    names_to = "measurement",
    values_to = "value"
  )
#> # A tibble: 6 × 3
#>   id    measurement value
#>   <chr> <chr>       <dbl>
#> 1 A     bp1           100
#> 2 A     bp2           120
#> 3 B     bp1           140
#> 4 B     bp2           115
#> 5 C     bp1           120
#> 6 C     bp2           125
```

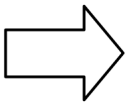
Let's visualize how this reshaping happens column by column. The values in a column which was already a variable in the original dataset (in this case `id`) need to be repeated, once for each column that is pivoted:



The left table represents what we originally typed when we created our dataset 'df.' The right table is what happened to the id when we used `pivot_longer()`. See how the data pivoted and repeated an `id` for each measurement?

Additionally, the original column names in `df` (`bp1` and `bp2`) now become values in a new variable, whose name is defined by the `names_to` argument and these values need to be repeated once for each row in the original dataset:

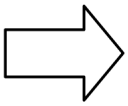
id	bp1	bp2
A	100	120
B	140	115
C	120	125



id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

And finally, the cell values also become a new variable with a name we defined by the `values_to` argument. These are unwound row by row:

id	bp1	bp2
A	100	120
B	140	115
C	120	125



id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

This is a general overview of pivoting but there are many more complex examples which are worth exploring in section 6.3.3 and 6.3.4 in the textbook.

2.5 Widening datasets

In less common cases, we may need to widen a dataset rather than lengthen it. Widening is essentially the opposite of lengthening and we do so by using the function `pivot_wider()`. `pivot_wider()` allows us to handle an observation if it is scattered across *multiple rows*. Let's

visualize this in the image below:

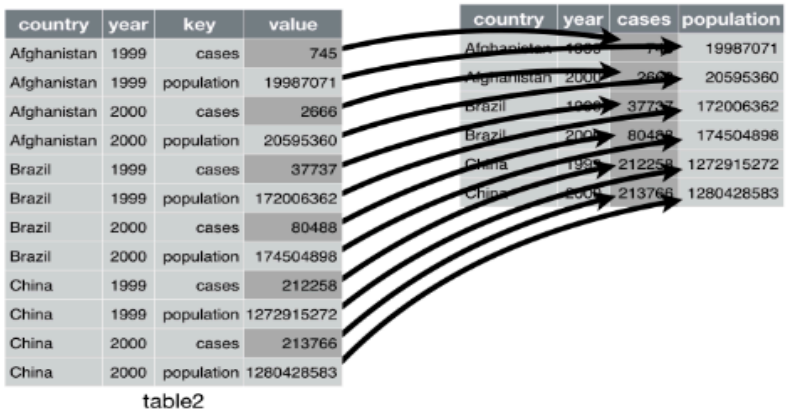


Figure 12.3: Pivoting `table2` into a "wider", tidy form.

In `table2` we see an observation is a country in a year with the observation spread across two rows (cases or population and their values). In this case, the table on the left needs to be made wider (like the table on the right) to move the value of cases into one column, and those of population into another column to comply with the 3 rules of tidy data (section **4.4**).

We'll use an example from R4DS to explore `pivot_wider()` looking at the `cms_patient_experience` dataset from the Centers of Medicare and Medicaid.

```
cms_patient_experience
#> # A tibble: 500 × 5
#>   org_pac_id org_nm                   measure_cd   measure_title
prf_rate
#>   <chr>      <chr>                    <chr>        <chr>
<dbl>
#> 1 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_1  CAHPS for MIPS...
63
#> 2 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_2  CAHPS for MIPS...
87
#> 3 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_3  CAHPS for MIPS...
86
#> 4 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_5  CAHPS for MIPS...
57
#> 5 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_8  CAHPS for MIPS...
85
#> 6 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_12 CAHPS for MIPS...
24
#> # ℹ 494 more rows
```

The core unit being studied is an organization. But in this format, each organization is spread across six rows with one row for each measurement taken in the survey organization. We can see the complete set of values for `measure_cd` and `measure_title` by using `distinct()`:

```
cms_patient_experience |>
  distinct(measure_cd, measure_title)
#> # A tibble: 6 × 2
#>   measure_cd   measure_title
#>   <chr>        <chr>
#> 1 CAHPS_GRP_1  CAHPS for MIPS SSM: Getting Timely Care, Appointments,
and In...
#> 2 CAHPS_GRP_2  CAHPS for MIPS SSM: How Well Providers Communicate
#> 3 CAHPS_GRP_3  CAHPS for MIPS SSM: Patient's Rating of Provider
#> 4 CAHPS_GRP_5  CAHPS for MIPS SSM: Health Promotion and Education
#> 5 CAHPS_GRP_8  CAHPS for MIPS SSM: Courteous and Helpful Office Staff
#> 6 CAHPS_GRP_12 CAHPS for MIPS SSM: Stewardship of Patient Resources
```

Neither of these columns will make particularly great variable names: `measure_cd` doesn't hint at the meaning of the variable and `measure_title` is a long sentence containing spaces. We'll use `measure_cd` as the source for our new column names for now, but in a real analysis you might want to create your own variable names that are both short and meaningful.

`pivot_wider()` has the opposite interface to `pivot_longer()`: instead of choosing new column names, we need to provide the existing columns that define the values (`values_from`) and the column name (`names_from`):

```
cms_patient_experience |>
  pivot_wider(
    names_from = measure_cd,
    values_from = prf_rate
  )
#> # A tibble: 500 × 9
#>   org_pac_id org_nm                  measure_title   CAHPS_GRP_1
CAHPS_GRP_2
#>   <chr>      <chr>                   <chr>                 <dbl>
<dbl>
#> 1 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          63
NA
#> 2 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          NA
87
#> 3 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          NA
NA
#> 4 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          NA
```

```
NA
#> 5 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          NA
NA
#> 6 0446157747 USC CARE MEDICAL GROUP ... CAHPS for MIPS...          NA
NA
#> # ℹ 494 more rows
#> # ℹ 4 more variables: CAHPS_GRP_3 <dbl>, CAHPS_GRP_5 <dbl>, ...
```

The above output doesn't look quite right; we still seem to have multiple rows for each organisation. That's because, we also need to tell `pivot_wider()` which column or columns have values that uniquely identify each row; in this case those are the variables starting with "`org`":

```
cms_patient_experience |>
  pivot_wider(
    id_cols = starts_with("org"),
    names_from = measure_cd,
    values_from = prf_rate
  )
#> # A tibble: 95 × 8
#>   org_pac_id org_nm           CAHPS_GRP_1 CAHPS_GRP_2 CAHPS_GRP_3
CAHPS_GRP_5
#>   <chr>      <chr>                  <dbl>       <dbl>       <dbl>
<dbl>
#> 1 0446157747 USC CARE MEDICA...          63          87          86
57
#> 2 0446162697 ASSOCIATION OF ...          59          85          83
63
#> 3 0547164295 BEAVER MEDICAL ...          49          NA          75
44
#> 4 0749333730 CAPE PHYSICIANS...          67          84          85
65
#> 5 0840104360 ALLIANCE PHYSIC...          66          87          87
64
#> 6 0840109864 REX HOSPITAL INC          73          87          84
67
#> # ℹ 89 more rows
#> # ℹ 2 more variables: CAHPS_GRP_8 <dbl>, CAHPS_GRP_12 <dbl>
```

And voila! This gives us the output we are looking for!

Pivoting wider - example

To understand what `pivot_wider()` does to our data, let's once again use a simple example. This time we have two patients with `id` s A and B, and we have three blood pressure (bp) measurements from patient A and two from patient B:

```
df <- tribble(
  ~id, ~measurement, ~value,
  "A",        "bp1",    100,
  "B",        "bp1",    140,
  "B",        "bp2",    115,
  "A",        "bp2",    120,
  "A",        "bp3",    105
)
```

We'll take the names from the `measurement` column using the `names_from()` argument and the values from the `value` column using the `values_from()` argument:

```
df |>
  pivot_wider(
    names_from = measurement,
    values_from = value
  )
#> # A tibble: 2 × 4
#>   id      bp1   bp2   bp3
#>   <chr> <dbl> <dbl> <dbl>
#> 1 A       100   120   105
#> 2 B       140   115    NA
```

To start the pivoting process, `pivot_wider()` needs to first figure out what will go in the rows and columns. The new column names will be the unique values of `measurement`.

```
df |>
  distinct(measurement) |>
  pull()
#> [1] "bp1" "bp2" "bp3"
```

By default, the rows in the output are determined by all the variables that aren't going into the new names or values. These are called the `id_cols`. Here there is only one column, but in general there can be any number.

```
df |>
  select(-measurement, -value) |>
```

```
  distinct()
#> # A tibble: 2 × 1
#>   id
#>   <chr>
#> 1 A
#> 2 B
```

`pivot_wider()` then combines these results to generate an empty dataframe:

```
df |>
  select(-measurement, -value) |>
  distinct() |>
  mutate(x = NA, y = NA, z = NA)
#> # A tibble: 2 × 4
#>   id    x     y     z
#>   <chr> <lgl> <lgl> <lgl>
#> 1 A     NA    NA    NA
#> 2 B     NA    NA    NA
```

It then fills in all the missing values using the data in the input. In this case, not every cell in the output has a corresponding value in the input as there's no third blood pressure measurement for patient B, so that cell remains missing. You can read about how `pivot_wider()` "makes" missing values in chapter 19 of the textbook. And we will cover missing values later in this workshop.

2.6 Pivoting Exercises using Palmer Penguins

Lengthening data with `pivot_longer()`

Imagine we want to create a single plot that shows the distribution of *all* morphometric measurements (bill length, bill depth, flipper length, and body mass) for our penguins. In its current tidy format, these variables are spread across four separate columns. To plot them together easily using `facet_wrap()`, we need them stacked into just two columns: one identifying the measurement name, and one holding the actual numerical value.

We can achieve this by gathering those columns into a "longer" format:

```
library(palmerpenguins)

# Create a long version of the penguins dataset

penguins_long <- penguins |>
  pivot_longer(
    cols = c(bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g),
    names_to = "measurement_type",
    values_to = "value"
  )
# View the result
head(penguins_long)
```

Have a look at the structure of this new tbl and notice the syntax: we specified the target columns we wanted to pivot (`cols`), the name of the new categorical column that will store our old column headers (`names_to`), and the name of the new numeric column that will house the data (`values_to`).

This lengthened format is exactly what `ggplot2` expects for multi-panel faceting. Let's visualise these distributions simultaneously:

```
penguins_long |>
  drop_na(value) |>
```

```
  ggplot(aes(x = value, fill = species)) +
  geom_histogram(bins = 30, alpha = 0.7, colour = "black") +
  facet_wrap(~ measurement_type, scales = "free_x") +
  theme_minimal() +
  labs(
    title = "Morphometric distributions across penguin species",
    x = "Measurement value",
    y = "Frequency"
  )
```

Widening data with `pivot_wider()`

Now, let's look at the reverse operation. Often in marine science literature, we need to present summary statistics as a cross-tabulated matrix. It is much easier for a reader to parse a wide table than a long, repetitive list.

First, let's calculate the mean body mass for each species on each island using `group_by()` and `summarise()`.

```
mass_summary <- penguins |>
  drop_na(body_mass_g) |>
  group_by(species, island) |>
  summarise(mean_mass = mean(body_mass_g))

head(mass_summary)
```

This output is technically "tidy" and computationally useful, but it is not ideal for publication presentation. We can use `pivot_wider()` to pull the island names up into columns, creating a clean matrix of species by island.

```
mass_matrix <- mass_summary |>
  pivot_wider(
    names_from = island,
    values_from = mean_mass
  )
```

```
head(mass_matrix)
```

Here, `names_from` dictates which column provides our new column headers (the islands), and `values_from` dictates which column populates the cells beneath them (the mean mass). You will notice `NA` values automatically populate where a specific species does not occur on a particular island—an immediate biological insight visualised directly through our data wrangling!

2.7 Separating and uniting data tables

We have seen so far how to tidy datasets by lengthening and widening them.

This section comes from the first edition of the R4DS textbook .In `table3,` we see one column (`rate`) that contains two variables (`cases and population`). To address this, we can use the `separate()` function which separates one column into multiple columns wherever you designate.

Here's another image from R4DS to help illustrate this:

country	year	rate
Afghanistan	1999	**745** / 19987071
Afghanistan	2000	**2666** / 20595360
Brazil	1999	**37737** / 172006362
Brazil	2000	**80488** / 174504898
China	1999	**212258** / 1272915272
China	2000	**213766** / 1280428583

country	year	cases	population
Afghanistan	1999	**745**	19987071
Afghanistan	2000	**2666**	20595360
Brazil	1999	**37737**	172006362
Brazil	2000	**80488**	174504898
China	1999	**212258**	1272915272
China	2000	**213766**	1280428583

table3

Here's `table3:`

```
table3
#> # A tibble: 6 × 3
#>   country      year rate
```

```
#> * <chr>       <int> <chr>
#> 1 Afghanistan  1999 745/19987071
#> 2 Afghanistan  2000 2666/20595360
#> 3 Brazil       1999 37737/172006362
#> 4 Brazil       2000 80488/174504898
#> 5 China        1999 212258/1272915272
#> 6 China        2000 213766/1280428583
```

We need to split the rate column up into two variables: 1) cases and 2) population. `separate()` will take the name of the column we want to split and the names of the columns we want it split *into.* See the code below:

```
table3 %>%
  separate(rate, into = c("cases", "population"))
#> # A tibble: 6 × 4
#>   country      year cases  population
#>   <chr>       <int> <chr>  <chr>
#> 1 Afghanistan  1999 745    19987071
#> 2 Afghanistan  2000 2666   20595360
#> 3 Brazil       1999 37737  172006362
#> 4 Brazil       2000 80488  174504898
#> 5 China        1999 212258 1272915272
#> 6 China        2000 213766 1280428583
```

Note: by default, separate() will split values wherever it sees a non-alphanumeric character (i.e. a character that isn't a number or letter). For example, in the code above, separate() split the values of rate at the forward slash characters. If you wish to use a specific character to separate a column, you can pass the character to the sep argument of separate(). For example, we could rewrite the code above as:

```
table3 %>%
  separate(rate, into = c("cases", "population"), sep = "/")
```

Notice the data types in `table3` above. Both `cases` and `population` are listed as character (`<chr>`) types. This is a default of using `separate()`. However, since the values in those columns are actually numbers, we want to ask `separate()` to convert them to better types using `convert = TRUE`. Now you can see they are listed as integer types(`<int>`)

```
table3 %>%
  separate(rate, into = c("cases", "population"), convert = TRUE)
#> # A tibble: 6 × 4
#>   country      year  cases population
```

```
#>   <chr>       <int>  <int>      <int>
#> 1 Afghanistan  1999    745   19987071
#> 2 Afghanistan  2000   2666   20595360
#> 3 Brazil       1999  37737  172006362
#> 4 Brazil       2000  80488  174504898
#> 5 China        1999 212258 1272915272
#> 6 China        2000 213766 1280428583
```

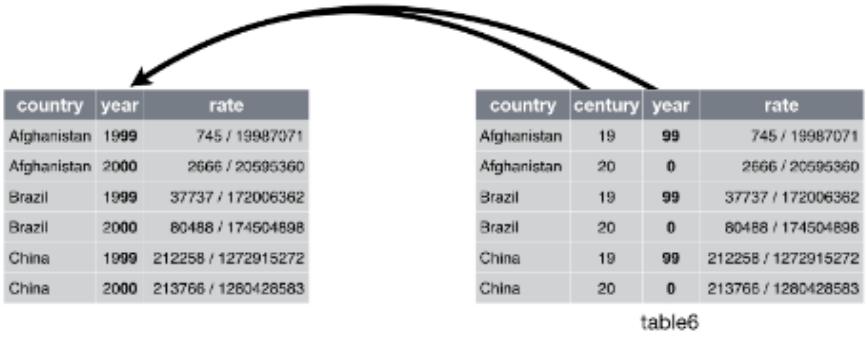
R4DS: You can also pass a vector of integers to sep. `separate()` will interpret the integers as positions to split at. Positive values start at 1 on the far-left of the strings; negative values start at -1 on the far-right of the strings. When using integers to separate strings, the length of sep should be one less than the number of names in into.

You can use this arrangement to separate the last two digits of each year. This makes this data less tidy, but is useful in other cases, as you'll see in a little bit.

```
table3 %>%
  separate(year, into = c("century", "year"), sep = 2)
#> # A tibble: 6 × 4
#>   country     century year  rate
#>   <chr>       <chr>   <chr> <chr>
#> 1 Afghanistan 19      99    745/19987071
#> 2 Afghanistan 20      00    2666/20595360
#> 3 Brazil      19      99    37737/172006362
#> 4 Brazil      20      00    80488/174504898
#> 5 China       19      99    212258/1272915272
#> 6 China       20      00    213766/1280428583
```

Using `unite()`:

To perform the inverse of `separate()` we will use `unite()` to combine multiple columns into a single column. In the example below for `table5`, we use `unite()` to rejoin `century` and `year` columns. `unite()` takes a data frame, the name of the new variable and a set of columns to combine using `dplyr::select()`.



country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

country	century	year	rate
Afghanistan	19	99	745 / 19987071
Afghanistan	20	0	2666 / 20595360
Brazil	19	99	37737 / 172006362
Brazil	20	0	80488 / 174504898
China	19	99	212258 / 1272915272
China	20	0	213766 / 1280428583

table6

Figure 12.5: Uniting `table5` makes it tidy

```
table5 %>%
  unite(new, century, year, sep = "")
#> # A tibble: 6 × 3
#>   country     new   rate
#>   <chr>       <chr> <chr>
#> 1 Afghanistan 1999  745/19987071
#> 2 Afghanistan 2000  2666/20595360
#> 3 Brazil      1999  37737/172006362
#> 4 Brazil      2000  80488/174504898
#> 5 China       1999  212258/1272915272
#> 6 China       2000  213766/1280428583
```

Here we need to add `sep =""` because we don't want any separator (the default is to add an underscore _)

2.8 Wrangling strings and dates

When we combine data from different sources—say, a government database, an oceanographic sensor, and handwritten field sheets—we inevitably run into formatting mismatches. A computer does not know that "Nelly Bay", "nelly_bay", and " NELLY BAY " are the same location. Similarly, date formats vary wildly depending on the instrument or the researcher's regional settings (e.g., DD/MM/YYYY vs. YYYY-MM-DD).

Before we can merge datasets or plot timelines, we must standardize our text strings and date-times. The `tidyverse` includes two specialized packages for this: `stringr` for text and `lubridate` for dates.

Standardising text with `stringr`

All functions in the `stringr` package begin with `str_`, making them easy to find using RStudio's autocomplete. Let's look at a common scenario: cleaning up a messy vector of site names before conducting a data summary.

```
library(tidyverse)

# A remarkably messy data frame of field sites
messy_sites <- tibble(
  site_id = c(" Nelly Bay", "nelly_bay", "NELLY BAY", " Geoffrey_Bay ", "geoffrey
bay")
)

# Using stringr within mutate to standardize the text
clean_sites <- messy_sites |>
  mutate(
    # 1. Convert everything to lowercase
    site_clean = str_to_lower(site_id),
    # 2. Replace any spaces with underscores
    site_clean = str_replace_all(site_clean, pattern = " ", replacement = "_"),
    # 3. Trim any leading or trailing whitespace (invisible spaces at the ends)
    site_clean = str_trim(site_clean)
  )

print(clean_sites)
```

By applying these three simple functions, we have converted five uniquely messy inputs into just two perfectly standardized categories: `"nelly_bay"` and `"geoffrey_bay"`.

Mastering time with `lubridate`

Base R is notoriously difficult when it comes to dates and times. The `lubridate` package simplifies this by allowing you to parse dates by simply spelling out the order of the components: year, month, day, hour, minute, and second.

If your dataset contains a date formatted as "25/12/2026", the day comes first, then the month, then the year. Therefore, you use the `dmy()` function. If the date is "2026-12-25", you use `ymd()`.

```
library(lubridate)

# Parsing different date formats
```

```
date_1 <- dmy("25/12/2026")
date_2 <- ymd("2026-12-25")

# R now recognizes these as identical Date objects
date_1 == date_2
```

For high-frequency marine data, such as acoustic telemetry or CTD profiles, we often need to parse exact timestamps. We do this by appending `_hms` (hours, minutes, seconds) to our functions.

```
# A tibble of raw sensor data with a messy character timestamp
sensor_data <- tibble(
  raw_time = c("14-05-2026 08:30:00", "14-05-2026 08:45:00", "14-05-2026
09:00:00"),
  temperature = c(24.5, 24.6, 24.4)
)

# Converting character strings to true POSIXct datetime objects
sensor_clean <- sensor_data |>
  mutate(
    true_time = dmy_hms(raw_time)
  )

print(sensor_clean)
```

Once a column is a recognised datetime object, `ggplot2` will automatically understand how to format the x-axis chronologically, and you can easily extract specific components (e.g., using `month(true_time)` to get the month integer).

2.9 Relational data: Joining tables

In robust data architecture, information is often split across multiple tables to reduce redundancy. For example, you might have a massive table of daily fish catch records that only lists a `site_code`, and a separate, smaller metadata table that contains the exact GPS coordinates and zoning status for those site codes.

To map or model this data, we must bring these tables together based on a common identifying column, known as a **key**. The `dplyr` package provides several mutating joins to accomplish this.

Let's set up two tables to demonstrate:

```
# Table 1: Biological observation data
observations <- tibble(
  site_code = c("NB", "GB", "MI", "NB", "HB"),
  species = c("Trout", "Snapper", "Trout", "Cod", "Trout"),
  count = c(5, 2, 1, 3, 8)
)

# Table 2: Spatial metadata
site_metadata <- tibble(
  site_code = c("NB", "GB", "MI", "RP", "WP"),
  zone = c("Marine National Park", "Conservation Park", "Habitat Protection",
"General Use", "Other Use"),
  lat = c(-19.16, -19.15, -19.14, -19.12, -19.11)
)
```

The workhorse: `left_ and right_join()`

The `left_join()` and `left_join()` functions are the most common and safest joins to use (they work the same way but in opposite directions in terms of how your code is written). For example `left_join()`keeps *all* the rows from your left table (`observations`) and matches data from your right table (`site_metadata`) wherever the key matches. If a match is missing, it fills the gaps with `NA`.

```
# Joining metadata to our observations
joined_data <- observations |>
  left_join(site_metadata, by = join_by(site_code))

print(joined_data)
```

Notice how every row of our biological data now has a zone and latitude securely attached to it? Cool!

The strict filter: `inner_join()`

An `inner_join()` only keeps rows that have complete matches in both tables. If an observation occurs at a site that isn't listed in the metadata, that observation will be entirely dropped from the dataset. Use this when you only want to analyze data that has complete spatial or environmental context.

```
matched_data <- observations |>
  inner_join(site_metadata, by = join_by(site_code))
glimpse(matched_data)
```

The detective: `anti_join()`

The `anti_join()` does not add new columns. Instead, it filters your left table to *only* show rows that do not have a match in the right table. This is an incredibly powerful diagnostic tool. If you try a `left_join()` and end up with unexpected `NA` values, use an `anti_join()` to find out exactly which site codes are misspelled or missing from your metadata table.

```
# Which observations are missing from our metadata dictionary?
missing_context <- observations |>
  anti_join(site_metadata, by = join_by(site_code))
```

2.10 Handling missing values

Missing data (`NA`) is an unavoidable reality in field science. Sensors fail, batteries die, and weather prevents sampling. Sometimes cells in a field datasheet are deliberately left blank to indicate zero values when time is of the essence. How we handle these missing values dictates the validity of our subsequent statistics. Furthermore, we can also encounter "fake" missing values generated by legacy equipment, or mathematical errors generated during our own calculations.

Converting legacy errors with `na_if()`

With some kinds of equipment, you can encounter data-outputs where a concrete value actually represents a missing value. This typically happens when data is generated from older oceanographic hardware (like legacy CTD casts or light loggers) that cannot properly output a blank `NA`. Instead, they spit out an impossible placeholder like `-99` or `-999` when a sensor errors out.

If you leave these in your dataset, your mean temperature calculations will be completely ruined. You can fix this securely inside a `mutate()` using `dplyr::na_if()`:

```
# Raw data from an old temperature logger
logger_data <- tibble(
  depth_m = c(10, 20, 30, 40),
  temp_c = c(24.5, 24.1, -999, 23.5)) # -999 is a known sensor error code

# Convert the -999 error codes to true NA values
```

```
fixed_logger <- logger_data |>
  mutate(temp_c = na_if(temp_c, -999))

print(fixed_logger)
```

(Note: It is always best practice to catch and convert these kinds of errors immediately after importing your data, before you attempt any visualisations).

Replacing missing values with `coalesce()`

Sometimes an `NA` does not mean the data is missing; it means the value is a known, fixed number—most commonly zero. For example, if a surveyor leaves a cell blank on a clipboard to indicate they saw zero sharks, R will import that blank cell as `NA`.

We can use `dplyr::coalesce()` inside a `mutate()` to hunt down every `NA` in a specific column and replace it with a `0` (or any other fixed value).

```
# Simulate some count data
shark_counts <- tibble(
  site = c("Reef_A", "Reef_B", "Reef_C"),
  shark_count = c(3, NA, 5)) # The NA here actually means 0 sharks were seen

# Replace NA with 0
shark_fixed <- shark_counts |>
  mutate(shark_count = coalesce(shark_count, 0))

print(shark_fixed)
```

(Bonus tip: `coalesce()` is incredibly powerful. If you have two columns, like a `primary_sensor` and a `backup_sensor`, `coalesce(primary_sensor, backup_sensor)` will automatically use the primary reading, but seamlessly fill in the gaps with the backup reading if the primary had failed!)

The special case of `NaN` (Not a Number)

One distinct type of missing value worth mentioning is `NaN` (Not a Number). While `NA` means "we do not know what this value is", `NaN` means "this value is mathematically impossible."

`NaN` is most common when you perform a mathematical operation that has an indeterminate result, such as dividing zero by zero. In marine science, this often happens when calculating

metrics like Catch Per Unit Effort (CPUE) for a site where both the catch and the effort were zero.

```
cpue_data <- tibble(
  site = c("Bay_1", "Bay_2"),
  catch = c(10, 0),
  effort_hours = c(2, 0)))

# Calculating CPUE (catch / effort)
cpue_calc <- cpue_data |>
  mutate(
    cpue = catch / effort_hours)

print(cpue_calc)
```

In the output, `Bay_2` will display `NaN`. Generally, R treats `NaN` identically to `NA` (for example, `drop_na()` will remove both). However, if you need to specifically audit your data for mathematical errors without flagging standard missing data, you can isolate them using the `is.nan()` function.

Implicit missing values and the zero-data framework: `complete()`

In community ecology, knowing where a species *was not* found is just as important as knowing where it *was* found. However, raw data almost exclusively records presences (implicit missing data).

Consider this raw catch log of coral trout species. It only lists what was caught:

```
raw_catch <- tibble(
  site = c("Reef_1", "Reef_1", "Reef_2"),
  species = c("Pmaculatus", "Pleopardus", "Pmaculatus"),
  count = c(5, 2, 8))
```

It is implied that Reef_2 was surveyed but that no *Plectopomus leopardus* were found there. If we calculate the mean catch of *P. leopardus* across all sites, base R will tell us it is 2 (because it only knows about Reef_1). Ecologically, the mean should be 1 (2 at Reef_1, and 0 at Reef_2).

We can use `tidyr::complete()` to build a zero-data framework. This function finds all unique combinations of the variables you supply, explicitly creates rows for the missing combinations, and allows you to fill the resulting `NA`s with `0`.

```
# Force inclusion of hidden zero-catch data
full_catch_matrix <- raw_catch |>
  complete(site, species, fill = list(count = 0))

print(full_catch_matrix)
```

Notice how `Reef_2` now explicitly lists `Pleopardus` with a count of `0`. Building these frameworks is a mandatory prerequisite before feeding data into Generalised Linear Models (GLMs) and other analyses.

The nuclear option: `drop_na()`

If an `NA` represents a failed instrument reading, and that reading is the core response variable of your analysis, you may have no choice but to remove the entire row. We can do this cleanly using `drop_na()`.

```
sensor_log <- tibble(
  day = 1:4,
  salinity = c(35.2, 35.1, NA, 35.3)
)

# Remove any row where salinity is missing
clean_log <- sensor_log |>
  drop_na(salinity)
```

2.11 Practical Exercises: Penguins, Pivots, and Relational Data

In this exercise, you will combine all the skills you have learned in this workshop. You have the clean `penguins` dataset, but a colleague has just emailed you a supplementary metadata file containing the GPS coordinates and the date the primary weather stations were installed on each island.

Unfortunately, your colleague is not very good at standardising their data entry.

Exercise 1: Cleaning the messy metadata

Run the following code to generate your colleague's messy metadata table in your environment:

```
# Make sure packages are loaded
library(tidyverse)
library(palmerpenguins)
```

```
# The messy metadata provided by your colleague
island_metadata <- tibble(
  island_name = c(" biscoe", "Dream ", "Torgersen"),
  station_install = c("15/01/2003", "22-03-2004", "05/11/2001"),
  latitude = c(-64.81, -64.73, -64.76)
)

print(island_metadata)
```

Task: Create a new object called `clean_metadata`. Using `mutate()`, `stringr`, and `lubridate`, fix the `island_name` column so that it perfectly matches the `island` column in the `penguins` dataset (hint: watch out for those hidden spaces!). Then, convert the `station_install` column into a proper Date object.

(Try this yourself before looking at the solution below!)

Exercise 2: The Relational Join

Now that you have a clean metadata table, it is time to merge it with your biological data.

Task: Create a new object called `penguins_spatial`. Use a `left_join()` to attach your `clean_metadata` to the main `penguins` dataset. *Hint: Because the column containing the island names is called* `island` *in the penguins dataset, and* `island_name` *in your metadata, you will need to specify how they match using* `by = join_by(island == island_name)`.

Check that it worked:

```
# Scroll to the right to see the new columns
head(penguins_spatial)
```

Exercise 3: The Wide Summary Matrix

A collaborator has asked for a clean, publication-ready table showing the **maximum body mass** recorded for each species, separated by island.

Task: Starting with your `penguins_spatial` dataset:

Drop any missing values in the `body_mass_g` column.

1. Group by `species` and `island`.
2. Summarise the data to find the maximum body mass (`max()`).

3. Pivot the dataset wider so that the species form the rows, and the islands form the columns.

2.12 Keystone exercise: Estuary Fish Survey - Data Rescue Mission

The Premise: You have inherited a project from a former PhD student who was studying the relationship between water quality and predatory fish assemblages along the Ross River Estuary gradient. Unfortunately, their data management was entirely manual, and the files are a disaster.

They left you with four separate datasets:

1. `estuary_catch_log.xlsx:` Handwritten field counts of fish species (using common names), spread across multiple spreadsheet tabs.
2. `estuary_metadata.csv`: The spatial coordinates and estuary zones for each site.
3. `estuary_sonde_data.csv`: High-frequency water quality sensor readings.
4. `species_dictionary.csv`: A taxonomic lookup table linking local common names to their accepted scientific names.

Your mission is to write a single, reproducible R script (or `.qmd`) that ingests this mess, cleans it, joins it into a single master dataset, and produces a publication-ready visualisation of the estuary's ecology.

Note: the datasets are now available in the class repository

Deliverables (to include in ePortfolio report/page)

At the end of this exercise, your final script/document must successfully output:

1. **A clean master dataset** containing no legacy errors `(-999.0)`, standardised text, a complete zero-catch framework, and scientific names instead of common names.
2. **A statistical summary table** displaying the mean and standard deviation of fish counts and salinity for each species per zone.
3. **A multi-panel plot** (faceted by species) showing fish abundance along the salinity gradient, formatted to professional publication standards.

Rules of Engagement

Because this is an open-ended task, there is no single "correct" way to approach this and structure your code. However, think about all the data-wrangling and visualisation skills you have developed over recent weeks, and adhere to the following professional standards:

- **Monitor your AI agents:** You are encouraged to use AI to help you write code. However, for every major data wrangling step, make sure you understand what is going on and are not just running AI-written code blindly.
- **Annotate your work:** To help you with the above, make sure you include comments (`#`) explaining what the code is doing. If you used an AI prompt to generate a complex block

of code, paste your prompt as a comment above it.

4. **Utilise the tidyverse:** There are many ways of wrangling data, but here we are looking for you to use your growing skills in `dplyr`, `tidyr`, `stringr`, and `lubridate` functions linked by the pipe (`|>` or `%>%`).

- **Reproducible workflow:** Make sure you stay on top of your PC → Git → GitHub lifecycle, by utilising: save, commit, pull, push!

Mission Objectives

Phase 1: Ingestion and Decontamination Load the four datasets. Before you attempt to join them, think about how you will need to wrangle and clean them individually:

- **Excel Tab Iteration:** The catch log is split across multiple tabs (one for each site). You will need to figure out how to read all tabs and bind them together into a single data frame.
- **String Standardisation:** The site and species names in the catch log are a mess (e.g., "MANGROVE JACK" vs "mangrove_jack"). Standardise all site and species columns to lowercase with underscores so they can securely match your dictionary and metadata tables later.
- **Temporal Parsing:** The dates in the sonde dataset are messy character strings. Parse them into formal Date-Time objects.
- **Hardware Errors:** The turbidity sensor occasionally failed and logged a value of `-999.0`. Identify these and convert them to true `NA` values before they ruin your averages.

Phase 2: The Relational Architecture Bring the data together into a cohesive format:

- **Summarise the water sensor data:** The sensor data is logged every 15 minutes, but your catch data is daily. Create a daily summary table that calculates the mean temperature and salinity for each site on each day. (Hint: look at the `floor_date()` function to group by day).
- **Taxonomic Translation:** Scientific journals will not accept "bream" or "flathead". Use a mutating join to merge your `species_dictionary` into your catch data, replacing the common names with accurate scientific taxonomy.
- **The Grand Join:** Merge your translated catch data, your spatial metadata, and your daily water quality summaries into a single master data frame.

Phase 3: The Zero-Catch Framework The former student only recorded a row when they actually caught a fish:

- Use `tidyr::complete()` to build a zero-data framework. Ensure that every species is represented at every site, on every sampling day.
- Replace the resulting `NA` values in the catch column with `0` using `coalesce()`.

Phase 4: Statistical Extraction From your master dataset, generate a clean summary table showing the mean ± standard error of both the fish catch and the salinity for each scientific

species across each estuary zone.

Phase 5: Visual Communication Prove that your data rescue was successful. Construct a multi-panel plot using `ggplot2` and `facet_wrap()`.

- **The Goal:** Visualise how the abundance of different species changes along the salinity gradient.
- **The Requirement:** The plot should have professional axis labels, a clean theme (e.g., `theme_minimal()`), be ordered spatially (e.g. from upstream to downstream), and the facet labels (the scientific names) should really be italicised!